Atelier 3 : Régression Logistique

1.DataSet	2.Model	3.Model Training	4.Model Evaluation

Dans cet atelier, utiliser le Dataset https://www.kaggle.com/saurabh00007/diabetescsv

Exercice 1 : Concepts de base

1. Donner une brève définition à un problème de classification ?
2. Comment peut-on distinguer entre un problème de régression et un problème de classification ?
3. Donner une brève définition à Logistic Regression
4. Quels sont les éléments mathématiques de Logistic Regression
5. Quels sont les paramètres du modèle en cas de simple logistic regression ?
6. Quels sont les paramètres du modèle en cas de multiple Logistic Regression ? donner l'écriture simplifiée du modèle (en utilisant les vecteurs et les matrices).

Avant de passer à l'exercice 2, les compétences suivantes en python sont requises (voir annexe – program1):

- Ouvrir un data set à l'aide de pandas et récupérer un dataframe
- Connaitre les dimensions du dataframe
- Explorer la liste des colonnes
- Récupérer une colonne, un ensemble de colonnes
- Récupérer un sous ensemble du dataframe (filtre)
- Le max, le min, la moyenne d'une colonne

Exercice 2 : Multiple Logistic Regression avec sklearn

Dans cet exercice, nous allons utiliser le dataset diabetescsv.csv (lien ci-dessus) pour faire la classification

1. Quelles sont les principales étapes à suivre pour établir un modèle de Simple Logistic Regression basé sur sklearn ?
2. **Data**
    a. Afficher le dataset (X,y) puis afficher les principales informations relatives au dataset ?
    b. Pourquoi il est nécessaire de standardiser les données ?
    c. À l'aide de train_test_split de sklearn.model_selection, diviser le dataset en training dataset et test dataset (80% pour training_data et 20% pour test_data)
    d. Pourquoi il faut séparer les données en données d'apprentissage et en données de test ?
    e. Afficher le nombre d'enregistrements pour le training et le nombre d'enregistrement pour le test
3. **Entrainement et utilisation du Modèle**
    a. Créer le modèle
    b. Faire l'apprentissage
    c. Afficher les paramètres du modèle obtenus après apprentissage
    d. En utilisant les paramètres trouvés par l'algorithme :
        i. Écrire l'expression mathématique pour faire la prédiction pour une personne de la base de test
        ii. Calculer la valeur (à l'aide de python) de cette personne et interpréter le résultat obtenu
        iii. À quelle classe appartient cette personne ?
    e. En utilisant les coefficients trouvés par l'algorithme :
        i. Écrire l'expression mathématique pour faire la prédiction pour un ensemble de personnes

ii. Calculer la sortie pour l'ensemble des valeurs (à l'aide de python) et interpréter le résultat obtenu
iii. À quelle classe appartient chaque personne ?

Exercice 3 : Simple Logistic Regression from scratch

1. Donner une brève définition à Gradient Descent
2. Quels sont les principaux éléments de Gradient Descent ? préciser le rôle de chaque élément

Avant de compléter l'exercice, la bonne compréhension du program1 et program2 en annexe est requise :

On reprend l'algorithme du program2 en annexe

```
def fit(X,y):
  n_samples,n_features=X.shape
  learning_rate=0.001
  n_iterations=500
  bias=0
  W=np.zeros(n_features)  #ligne 6
  for _ in range(n_iterations):
    y_predicted=np.dot(X,weights)+bias #ligne 8
    d_W=(2/n_samples)*np.dot(X.T,(y_predicted-y)) #ligne 9
    d_bias=(2/n_samples)*np.sum(y_predicted-y)  #ligne 10
    W-= learning_rate *d_W  #ligne 11
    bias-= learning_rate *d_bias  #ligne 12
  return bias,W
```

1. Après analyse du code, quelles sont les modifications à apporter à l'algorithme pour qu'il prenne en compte la classification basée sur la régression logistique ?
2. Ecrire le code de la fonction sigmoid (LinearModel) permettant de retourner un vecteur de valeurs ?
3. Que représente le vecteur retourné par sigmoid ?
4. Que représente LinearModel, le paramètre de la fonction sigmoid ?
5. Ajouter une fonction predict (X) permettant de retourner les classes des lignes de X
6. Organiser le code obtenu sous forme d'une classe LogisticReg avec les éléments suivants :
   - Les hyperparametrs du modèle: learning_rate, epochs
   - Les paramètres du modèle : bias, weights
   - La fonction fit : pour faire l'apprentissage
   - La fonction predict : pour faire la prédiction

Dans la suite, nous allons utiliser le dataset diabetescsv.csv (lien ci-dessus) pour faire la classification

7. Data
   - Charger les données à l'aide de pandas
   - Afficher le min, max, moyenne de chaque colonne
   - Définir X et y
8. Model
   - On reprend la fonction fit trouvée en exercice 3
   - Quelles sont les modifications à apporter à cet algorithme pour l'adapter à ce nouveau contexte ? justifier ?
   - Quel est la taille de W dans ce cas
   - Pour un nombre d'itération 1000 et un learning_rate=0.0001, écrire l'expression de la fonction logit et l'expression de la fonction logistique ?
9. Evaluation du modèle

- Evaluer le modèle avec les paramètres précédents (Test=20%)
- Pour les mêmes variables indépendantes, trouver le modèle à l'aide de sklearn
- Comparer la performance du modèle trouvé en 2, et celui de sklearn

Exercice 4 : Evaluation du modèle

1. Pourquoi il est important d'évaluer la performance du modèle ?
2. Que représente la matrice de confusion ? quels sont les éléments de la matrice de confusion en cas d'
3. Une classification binaire et en cas d'une multinomial classification?

Soit la matrice de confusion suivante :

	Predicted 0	Predicted 1
Actual 0	30	5
Actual 1	8	40

Quelles sont les valeurs de FP, TP, FN, TN

4. Soit les deux vecteurs suivants :

y_real	y_predicted
0	**0**
0	**0**
0	**0**
0	**1**
1	**1**
1	**0**
1	**1**
1	**1**
1	**0**
1	**1**

   a. Trouver la matrice de confusion équivalente aux vecteurs ci-dessus.
   b. Écrire une fonction en python qui retourne une matrice de confusion. Tester cette fonction en utilisant les deux vecteurs de la question précédente.
5. Matrice de confusion du modèle
   a. À l'aide de sklearn.metrics, afficher la matrice de confusion du modèle trouvé en Exercice 3-8)
   b. Retourner la même matrice en utilisant votre fonction
6. Que signifie accuracy metric ?

   La formule de accuracy : $acuracy = \frac{TP+TN}{TP+TN+FP+FN}$ pour une classification binaire

   a. Pour la matrice de confusion en 4.c, calculer accuracy du modèle trouvé en Exercice 3-8)
   b. À l'aide de sklearn.metrics.accuracy_score, calculer accuracy du modèle trouvé en Exercice 3-8)

   https://scikit-learn.org/stable/modules/generated/sklearn.metrics.accuracy_score.html
7. Que représente precision metric ? quelle est son utilité ? et comment l'interpréter pour connaitre la performance du modèle ?

La formule de precision : $precision = \frac{TP}{TP+FP}$

Calculer precision du modèle trouvé en Exercice 3-8)

8. Que représente recall metric ? ? quelle est son utilité ? et comment l'interpréter pour connaitre la performance du modèle ?

   La formule de recall : $recall = \frac{TP}{TP+FN}$

   Calculer recall du modèle trouvé en 3)

9. Que représente le F1-score ? quelle est son utilité ? et comment l'interpréter pour connaitre la performance du modèle ?

   La formule de F1-score : $f1_{score} = 2 * \frac{precision * recall}{precision+recall}$

10. Calculer les mesures (precision, recall, fscore) du modèle trouvé en 3) en utilisant

    precision_recall_fscore_support de sklearn.metrics

    interpreter les valeurs obtenus.

Annexe -program1

```
import numpy as np
from sklearn.model_selection import train_test_split
import pandas as pd
df=pd.read_csv("diabetes.csv")
cols=df.columns
print(df[['Glucose']].max())
#convertir le dataframe en array
data=np.array(df)
#extraire la partie features
X=np.array([data[i,:len(cols)] for i in range(len(data))])
#extraire la partie target
y=np.array([data[i,3] for i in range(len(data))])
#
from sklearn.preprocessing import StandardScaler
scaler= StandardScaler()
X = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=44) #
#random_state=valeur, permet de générer le même dataset à chaque fois. cette valeur assure d'avoir le même random
```

Annexe -program2 – fit pour LinearRegression

```
def fit(X,y):
   n_samples,n_features=X.shape
   learning_rate=0.00001
   n_iterations=1000
   bias=0
   W=np.zeros(n_features)  #ligne 6
   for _ in range(n_iterations):
      y_predicted=np.dot(X,weights)+bias #ligne 8
      d_W=(2/n_samples)*np.dot(X.T,(y_predicted-y)) #ligne 9
      d_bias=(2/n_samples)*np.sum(y_predicted-y)  #ligne 10
      W-= learning_rate *d_W  #ligne 11
      bias-= learning_rate *d_bias  #ligne 12
  return bias,W
bias,W=fit(X_train,y_train)
```

Les compétences suivantes sont requises :

- Interpréter le programme ligne par ligne
- Écrire le résultat de la ligne 6, pourquoi il faut utiliser un vecteur à la place d'un scalar.
- Écrire le résultat de la ligne 8, pourquoi il s'agit d'un vecteur à une seule dimension ? vérifier le résultat pour les deux premières itérations.
- Écrire le résultat de la ligne 9, pourquoi il s'agit d'un vecteur? vérifier le résultat pour les deux premières itérations.
- Écrire le résultat de la ligne 10, pourquoi il s'agit d'un scalar ? vérifier le résultat pour les deux premières itérations. Et pourquoi on a utilisé uniquement np.sum à la place de np.dot ?
- Écrire le résultat de la ligne 11, pourquoi il s'agit d'un vecteur ? vérifier le résultat pour les deux premières itérations.